

Playwright for Modern Test Automation

- What Playwright is and why teams are adopting it fast
- Core concepts of Playwright every automation engineer must know
- Smart locator strategies for stable tests
- Framework Design Insights
- Real time Interview Questions



AKSHAYA PANIGRAHI

Let's Deep Dive into Playwright

SWIPE →

Playwright for Modern Test Automation

What Playwright is ?

- Playwright is a modern automation framework that allows testers to automate web UI + API + cross-browser testing using a single tool.
- It directly communicates with browser engines (Chromium, Firefox, WebKit) → making tests faster, stable, and closer to real user behavior

Why Teams Are Adopting Playwright Fast ?

- **Built for modern web apps:** Handles dynamic UI, async loading, fewer flaky tests compared to traditional Selenium setups.
- **Auto-waiting :** No manual waits → Playwright automatically waits for elements to be ready → reduces maintenance effort
- **UI + API testing in one framework :** Supports end-to-end validation across layers → useful for service validation and failure handling
- **Parallel Executions Built In :** Teams achieve faster pipeline — important for CI/CD environments .
- **Cross-browser from single codebase :** Run the same test on Chromium, Firefox, and WebKit without changing scripts

Let's Deep Dive more into Playwright

SWIPE →

Core concepts of Playwright every automation engineer must know

- Browser → Context → Page Architecture

Playwright creates isolated browser contexts (like incognito sessions) for clean, parallel test execution essential for scalable enterprise frameworks.

- Auto-Waiting Mechanism

Playwright automatically waits for elements to be visible, stable, and actionable → reduces flaky tests and removes manual waits.

- Locator Strategy

Role, label, text, and test-id based locators improve stability and maintainability compared to traditional selectors.

- Built-in Parallel Execution

Tests run concurrently across browsers and environments → faster feedback in CI/CD pipelines.

- API Testing Support

Playwright supports backend validation using `APIRequestContext` → enables end-to-end workflow testing (API + UI together).

- Powerful Debugging Tools

Trace viewer, screenshots, videos, and network logs help quickly identify root causes of failures

Let's Deep Dive more into Playwright

SWIPE →

Smart locator strategies for stable tests

- Prefer user-visible attributes

Use role, label, placeholder, and text as they reflect real user interaction and remain stable across UI

- Follow locator priority hierarchy

Role / Label → TestId → Text → CSS → XPath

This keeps tests readable, maintainable, and less fragile in large automation suites

- Use accessibility-based locators

ARIA roles and accessible names improve reliability and indirectly validate accessibility compliance

- Leverage locator chaining for dynamic UI

Find parent container → filter by business text → act on child element.
This handles repeating components and dynamic content reliably.

- Use data attributes for framework-level stability

data-testid or business specific attributes provide consistent identification across environments.

- Centralize locator definitions

Store selectors in Page Objects to simplify maintenance and support scalable framework design

Sample HTML page for Practicing Locator

```
<!DOCTYPE html>
<html>
<head>
  <title>Playwright Locator Demo</title>
</head>
<body>

<h1>Login Page</h1>
<label for="username">Username</label>
<input id="username" type="text" placeholder="Enter username" />

<label for="password">Password</label>
<input id="password" type="password" placeholder="Enter password" />
<button id="loginBtn" data-testid="login-button">Login</button>
<a href="/register">Create Account</a>

<div class="product" data-product-id="101">
  <span class="product-name">Laptop</span>
  <button class="add-to-cart">Add to Cart</button>
</div>
</body>
</html>
```

Let's Deep Dive more into Locator

SWIPE →

Playwright Locator Examples (C#)

- By Role (Most Recommended)

```
await page.GetByRole(AriaRole.Button, new() { Name = "Login" }).ClickAsync();  
await page.GetByRole(AriaRole.Link, new() { Name = "Create Account" }).ClickAsync();
```

- By Label

```
await page.GetByLabel("Username").FillAsync("Akshaya");  
await page.GetByLabel("Password").FillAsync("12345");
```

- By Placeholder

```
await page.GetByPlaceholder("Enter username").FillAsync("Akshaya");
```

- By TestId

```
await page.GetByTestId("login-button").ClickAsync();
```

- By Text

```
await page.GetByText("Login").ClickAsync();  
await page.GetByText("Laptop").IsVisibleAsync();
```

- By Alt Text

```
await page.GetByAltText("Company Logo").IsVisibleAsync();
```

Playwright Locator Examples (C#)

- By TestId

```
await page.GetByTestId("login-button").ClickAsync();
```

- By CSS Locator

```
await page.Locator("#loginBtn").ClickAsync();
```

```
await page.Locator(".add-to-cart").ClickAsync();
```

- Locator Chaining (Dynamic UI handling)

```
var product = page.Locator(".product", new() { HasText = "Laptop" });
```

```
await product.Locator(".add-to-cart").ClickAsync();
```

Locator Priority Order

→ Role / Label

→ TestId

→ Text

→ AltText

→ CSS

→ XPath

Framework Design Insights

- Page Object Model for scalability

Encapsulate UI behavior inside page classes.

→ Reduces duplication and simplifies updates when UI changes

- Factory pattern for browser and context management

Centralized control of browser creation, environment setup, and test isolation.

→ Enables cross-browser and parallel execution smoothly

- Reusable fixtures for setup and teardown

Shared initialization logic ensures clean test state and predictable execution.

- Integrated UI + API workflow support

Framework supports test data setup via API and validation via UI.

→ Faster, more reliable end-to-end testing

- Centralized locator strategy

Store locators within page classes using user-centric selectors.

→ Reduces flaky tests and simplifies maintenance

- Configuration-driven execution

Environment URLs, browser selection, timeouts, and credentials managed via config files. Same tests run across QA, staging, and production-like setups.

Real time Interview Questions

- What problem does Playwright solve compared to Selenium?
 - Direct browser control → faster + stable
 - Auto-waiting → less flaky tests
 - Built-in parallel execution
 - UI + API automation in one framework
 - Cross-browser without drivers
- Explain Browser → Context → Page architecture
 - Browser → actual browser instance
 - Context → isolated session (like incognito)
 - Page → tab interaction layer
- What is auto-waiting in Playwright?
 - Playwright waits automatically for:
 - element visible
 - stable
 - enabled
 - attached

What locator strategy do you prefer in Playwright?

Role / Label → TestId → Text → CSS → XPath

Real time Interview Questions

- How do you handle dynamic elements?

- Locator chaining
- Parent-child filtering
- Business-text filtering

Ex: `var product = page.Locator(".product", new() { HasText = "Laptop" });`
`await product.Locator(".add-to-cart").ClickAsync();`

- Explain your Playwright framework architecture.

- ✓ Page Object Model
- ✓ Factory pattern for browser
- ✓ Config-driven execution
- ✓ Reusable fixtures
- ✓ CI/CD integration
- ✓ Cross-browser execution

- Can Playwright test APIs?

Yes, using `APIRequestContext`.

- How do you combine API and UI testing?

- Create test data via API
- Launch UI
- Validate business flow

Real time Interview Questions

- How do you debug failed tests?
 - ✓ Trace viewer
 - ✓ Screenshots
 - ✓ Video
 - ✓ Logs
 - ✓ Network monitoring
- Why are Playwright tests less flaky?
 - ✓ auto-waiting
 - ✓ stable locators
 - ✓ isolated contexts
 - ✓ direct browser control
- Test fails in CI but passes locally. What will you check?
 - Environment difference
 - Timing issues
 - Data dependency
 - Headless vs headed behavior
 - Network delays
- How do you handle multiple tabs?

Listen for new page event → switch context.

Real time Interview Questions

- How do you send an API request using Playwright?

using Microsoft.Playwright;

```
.  
  
var playwright = await Playwright.CreateAsync();  
var request = await playwright.APIRequest.NewContextAsync();  
  
var response = await  
request.GetAsync("https://jsonplaceholder.typicode.com/posts/1");  
Console.WriteLine(await response.TextAsync());
```

- How do you set Base URL for API testing?

```
var request = await playwright.APIRequest.NewContextAsync(  
    new APIRequestNewContextOptions  
    {  
        BaseURL = "https://jsonplaceholder.typicode.com"  
    });
```

```
var response = await request.GetAsync("/posts/1");
```

Real time Interview Questions

- How do you send POST request with JSON body?

```
var response = await request.PostAsync("/posts", new()  
{  
    DataObject = new  
    {  
        title = "Akshaya Test",  
        body = "Playwright API automation",  
        userId = 1  
    }  
});
```

- How do you send headers in API request?

```
var request = await playwright.APIRequest.NewContextAsync(  
    new()  
    {  
        ExtraHTTPHeaders = new Dictionary<string, string>  
        {  
            ["Authorization"] = "Bearer token123",  
            ["Content-Type"] = "application/json"  
        }  
    }  
});
```

Real time Interview Questions

- How do you send query parameters?

```
var response = await request.GetAsync("/posts", new()  
{  
    Params = new Dictionary<string, object>  
    {  
        ["userId"] = 1  
    }  
});
```

- How do you reuse authentication session?

```
await request.StorageStateAsync(new() { Path = "auth.json" });  
await playwright.APIRequest.NewContextAsync(  
    new() { StorageStatePath = "auth.json" });
```

- How do you validate response body?

```
var response = await request.GetAsync("/posts/1");  
  
string responseBody = await response.TextAsync();  
  
Assert.IsTrue(responseBody.Contains("userId"));  
Assert.IsTrue(responseBody.Contains("title"));
```